Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №2
на тему
«ЭЛЕМЕНТЫ КРИПТОГРАФИИ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
Вывод 6
Приложение А (Обязательное) Листинг программного кода 7

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является получение практических навыков в области классической криптографии путём самостоятельной реализации симметричных шифров замены. Изучение подобных алгоритмов позволяет сформировать базовое понимание принципов защиты информации, оценить стойкость простых шифров и осознать их уязвимости. В качестве объектов реализации выбраны шифр Цезаря и шифр Виженера.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Шифр Цезаря основан на сдвиге каждого символа текста на фиксированное число позиций вдоль алфавита. Числовой ключ K задаёт величину этого сдвига: при шифровании каждая буква заменяется буквой, стоящей на K позиций правее, а при выходе за границу алфавита счёт продолжается с его начала. Дешифрование выполняется обратным сдвигом на то же значение. Поскольку один и тот же ключ применяется ко всему тексту, шифр относится к классу моноалфавитных подстановок и уязвим к частотному анализу. Пример шифрования и дешифрования текста с помощью шифра Цезаря представлен на рисунке 2.1.

```
velebes@arch ... (main)> go run main.go
Choose encrypt or decrypt:
1. Encrypt
2. Decrypt
3. Exit
1
Enter name of file which contains text to encrypt:
input.txt

Choose cipher type:
1. Caesar
2. Vigenere
1

Enter key (positive number):
47
Operation completed. Result stored in input.txt

Choose encrypt or decrypt:
1. Encrypt
2. Decrypt
3. Exit
2
Enter name of file which contains text to decrypt:
input.txt

Choose cipher type:
1. Caesar
2. Vigenere
1

Enter key (positive number):
47
Operation completed. Result stored in input.txt
```

```
velebes@arch ... (main)> cat input.txt
Текст для зашифровки зашифруй меня
velebes@arch ... (main)> cat input.txt
Атшяа сщм хнёцвюьпшц хнёцвюбч ътым
velebes@arch ... (main)> cat input.txt
Текст для зашифровки зашифруй меня
velebes@arch ... (main)>
```

Рисунок 2.1 – Результат шифрования и дешифрования методом Цезаря

Шифр Виженера является полиалфавитным обобщением шифра Цезаря. Вместо одного числового ключа используется ключевое слово, каждая буква которого задаёт индивидуальный сдвиг для соответствующей буквы текста. Ключевое слово циклически накладывается на текст: буквы ключевого слова поочерёдно применяются к буквам текста, и после последней буквы ключа счёт начинается заново. Счётчик позиции в ключе продвигается только при обработке литерных символов, благодаря чему знаки препинания и пробелы не нарушают выравнивание ключа. В результате одна и та же буква в разных по-

зициях шифруется по-разному, что существенно усложняет частотный анализ. Пример работы с шифром Виженера показан на рисунке 2.2.

```
velebes@arch ... (main)> go run main.go
Choose encrypt or decrypt:
1. Encrypt
2. Decrypt
3. Exit
1
Enter name of file which contains text to encrypt:
input.txt

Choose cipher type:
1. Caesar
2. Vigenere
2

Enter key word:
abobakey
Operation completed. Result stored in input.txt

Choose encrypt or decrypt:
1. Encrypt
2. Decrypt
3. Exit
2
Enter name of file which contains text to decrypt:
input.txt

Choose cipher type:
1. Caesar
2. Vigenere
2

Enter key word:
abobakey
Operation completed. Result stored in input.txt
```

```
velebes@arch ... (main)> cat input.txt
Текст для зашифровки зашифруй меня
velebes@arch ... (main)> cat input.txt
Ужщуу орч ивжкхыуълк цвщущифл ыжой
velebes@arch ... (main)> cat input.txt
Текст для зашифровки зашифруй меня
velebes@arch ... (main)>
```

Рисунок 2.2 – Результат шифрования и дешифрования методом Виженера

Программа реализована в виде консольного приложения. Логика шифрования вынесена в отдельные функции: вспомогательная процедура определяет принадлежность символа к латинскому или русскому алфавиту (строчному или прописному) и выполняет циклический сдвиг с сохранением регистра, нелитерные символы при этом остаются неизменными. На основе этой процедуры построены независимые функции для шифра Цезаря и шифра Виженера. Счётчик позиции в ключе для шифра Виженера продвигается только при обработке литерных символов, что обеспечивает корректное выравнивание ключа при наличии пробелов и знаков препинания. Главный модуль в цикле предлагает пользователю выбрать режим работы (шифрование или дешифрование), ввести имя файла, выбрать алгоритм и указать ключ, результат записывается обратно в тот же файл.

ВЫВОД

В ходе выполнения лабораторной работы реализованы шифры Цезаря и Виженера на языке *Go* с поддержкой русского и латинского алфавитов, сохранением регистра и нелитерных символов. Разработан консольный интерфейс для выбора файла и режима работы. Тестирование подтвердило корректность обоих алгоритмов: последовательное шифрование и дешифрование полностью восстанавливает исходный текст.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Код программы

```
package main

import (
  "fmt"
  "io"
  "os"
)

const (
  enLower = "abcdefghijklmnopqrstuvwxyz"
  enUpper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
  ruLower = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"
  ruUpper = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
)

func shiftRune(r rune, k int) rune {
  findShift := func(alphabet string) rune {
    runes := []rune(alphabet)
    n := len(runes)
    for i, a := range runes {
      if r == a {
        newID := (i + k) % n
        if newID < 0 {
          newID += n
        }
        return runes[newID]
      }
    }
    return r
  }

  switch {
  case (r >= 'a' && r <= 'z'):
    return findShift(enLower)
  case (r >= 'A' && r <= 'Z'):
    return findShift(enUpper)
  case (r >= 'a' && r <= 'я') || r == 'ё':
    return findShift(ruLower)
  case (r >= 'A' && r <= 'Я') || r == 'Ё':
    return findShift(ruUpper)
  default:
    return r
  }
}

func Caesar(str []rune, key int) []rune {
  res := make([]rune, len(str))
  for i, c := range str {
    res[i] = shiftRune(c, key)
```

```
  }
  return res
}

func Vigenere(str []rune, key []rune, decrypt bool) []rune {
  res := make([]rune, len(str))
  keyID := 0
  for i, r := range str {
    var shift int
    var found bool

    k := key[keyID%len(key)]

    for i, a := range []rune(enLower) {
      if a == k || []rune(enUpper)[i] == k {
        shift = i
        found = true
        break
      }
    }
    if !found {
      for i, a := range []rune(ruLower) {
        if a == k || []rune(ruUpper)[i] == k {
          shift = i
          found = true
          break
        }
      }
    }

    shift++

    if decrypt {
      shift = -shift
    }

    newR := shiftRune(r, shift)
    res[i] = newR

    if newR != r {
      keyID++
    }
  }
  return res
}

func main() {
  var fileToRead string
  var res []rune

  var choice byte
  for {
    decrypt := false
    for {
      fmt.Println("Choose encrypt or decrypt:")
      fmt.Println("1. Encrypt")
      fmt.Println("2. Decrypt")
```

```
      fmt.Println("3. Exit")

      fmt.Scan(&choice)
      switch choice {
      case 1:
        fmt.Println("Enter name of file which contains text to
encrypt:")
      case 2:
        fmt.Println("Enter name of file which contains text to
decrypt:")
        decrypt = true
      case 3:
        return
      default:
        fmt.Println("Incorrect choice")
        continue
      }
      break
    }
    fmt.Scan(&fileToRead)

    file, err := os.Open(fileToRead)
    if err != nil {
      fmt.Printf("error opening file: %s\n", err)
      continue
    }

    content, err := io.ReadAll(file)
    if err != nil {
      fmt.Printf("error reading file: %s\n", err)
      file.Close()
      continue
    }
    file.Close()
    input := []rune(string(content))

    for {
      fmt.Println("\nChoose cipher type: ")
      fmt.Println("1. Caesar")
      fmt.Println("2. Vigenere")
      fmt.Scan(&choice)
      fmt.Println()

      switch choice {
      case 1:
        fmt.Println("Enter key (positive number):")
        var key int
        fmt.Scan(&key)
        if decrypt {
          key = -key
        }
        res = Caesar(input, key)
      case 2:
        fmt.Println("Enter key word:")
        var key string
        fmt.Scan(&key)
        res = Vigenere(input, []rune(key), decrypt)
```

```
      default:
        fmt.Println("Incorrect choice")
        continue
      }
      break
    }

    err = os.WriteFile(fileToRead, []byte(string(res)), 0644)
    if err != nil {
      fmt.Printf("error writing file: %s\n", err)
    }
    fmt.Printf("Operation completed. Result stored in %s\n\n",
fileToRead)
  }
}
```